

Analyzing Systems Using Data Dictionaries

LEARNING OBJECTIVES

Once you have mastered the material in this chapter, you will be able to:

1. Understand how analysts use data dictionaries for analyzing data-oriented systems.
2. Understand the concept of a repository for analysts' project information and the role of computer-aided software engineering (CASE) tools in creating them.
3. Create data dictionary entries based on data flow diagrams (DFDs) for data processes, stores, flows, structures, and logical and physical elements of the systems being studied.
4. Recognize the functions of data dictionaries in helping users update and maintain information systems.

When successive levels of data flow diagrams (DFDs) are complete, systems analysts use them to help catalog the data processes, flows, stores, structures, and elements in a data dictionary. Of particular importance are the names used to characterize data items. When given an opportunity to name components of data-oriented systems, a systems analyst needs to work at making names meaningful but exclusive of other existing data component names. This chapter covers the data dictionary, which is another tool that aids in the analysis of data-oriented systems.

A data dictionary is a specialized application of the kinds of dictionaries used as references in everyday life. A data dictionary is a reference work of data about data (that is, *metadata*). Systems analysts compile data dictionaries to guide them through analysis and design. A data dictionary is a document that collects and coordinates specific data terms, and it confirms what each term means to different people in the organization.

Automated data dictionaries (part of the CASE tools) are valuable for their capacity to cross-reference data items, thereby allowing necessary program changes to all applications that share a common element. This feature supplants changing programs on a haphazard basis, and it prevents waiting until the program won't run because a change has not been implemented across all programs sharing the updated item. Clearly, automated data dictionaries are important for large systems that produce several thousand data elements requiring cataloging and cross-referencing.

The Data Dictionary

Many database management systems now come equipped with an automated data dictionary. These dictionaries can be either elaborate or simple. Some computerized data dictionaries automatically catalog data items when coding is done; others simply provide a template to prompt the person filling in the dictionary to do so in a uniform manner for every entry.

One important reason for maintaining a data dictionary is to keep clean data. This means that data must be consistent. If you store data about a male's sex as "M" in one record, "Male" in a second record, and as the number "1" in a third record, the data are not clean. Keeping a data dictionary will help in this regard.

Need for Understanding the Data Dictionary

Despite the existence of automated data dictionaries, a systems analyst should understand what data compose a data dictionary, the conventions used in data dictionaries, and how a data dictionary is developed. Understanding the process of compiling a data dictionary can aid a systems analyst in conceptualizing the system and how it works. The data flow diagrams (DFDs) in Chapter 7 are an excellent starting point for collecting data dictionary entries.

The following sections describe the rationale behind contents of automated data dictionaries. In addition to providing documentation and eliminating redundancy, a data dictionary may be used to:

1. Validate the data flow diagram for completeness and accuracy.
2. Provide a starting point for developing screens and reports.
3. Determine the contents of data stored in files.
4. Develop the logic for data flow diagram processes.
5. Create extensible markup language (XML).

The Data Repository

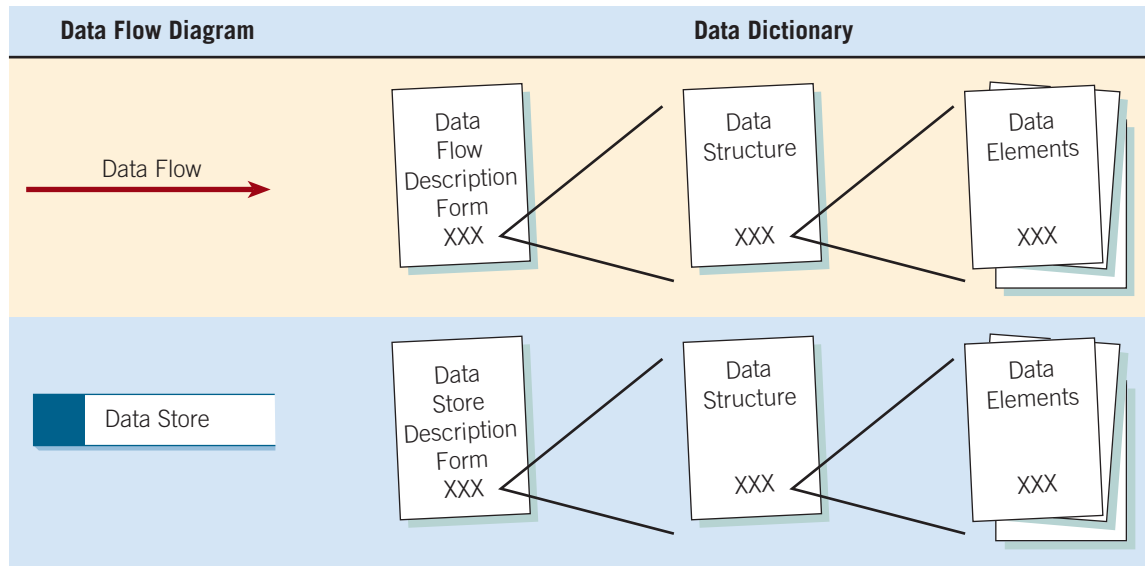
A data dictionary contains information about data and procedures; a larger collection of project information is called a *repository*. One of the benefits of using a CASE tool to develop the data dictionary is the ability to develop a repository, or a shared collection of project information and team contributions. The repository may contain the following:

1. Information about the data maintained by the system, including data flows, data stores, record structures, elements, entities, and messages
2. Procedural logic and use cases
3. Screen and report design
4. Data relationships, such as how one data structure is linked to another
5. Project requirements and final system deliverables
6. Project management information, such as delivery schedules, achievements, issues that need resolving, and project users

The data dictionary is created by examining and describing the contents of the data flows, data stores, and processes, as illustrated in Figure 8.1. Each data store and data flow should be defined and then expanded to include the details of the elements it contains. The logic of each process should be described using the data flowing into or out of the process. Omissions and other design errors should be noted and resolved.

The four data dictionary categories—data flows, data structures, data elements, and data stores—should be developed to promote understanding of the data in the system. Procedural logic is presented in Chapter 9, entities are discussed in Chapter 13, and messages and use cases are presented in Chapters 2 and 10.

To illustrate how data dictionary entries are created, we use an example for World's Trend Catalog Division. This company sells clothing and other items by mail order, by a toll-free phone order system (or faxing the mail order form), and online using customized Web forms. Regardless of the origin of the order, the underlying data captured by the system are the same for all three methods.

**FIGURE 8.1**

How data dictionaries relate to data flow diagrams.

The World's Trend order form shown in Figure 8.2 gives some clues about what to enter into a data dictionary. First, you need to capture and store the name, address, and telephone number of the person placing the order. Then you need to address the details of the order: the item description, size, color, price, quantity, and so on. The customer's method of payment must also be determined. Once you have done this, these data may be stored for future use. This example is used throughout this chapter to illustrate each part of the data dictionary.

Defining the Data Flows

Data flows are usually the first components to be defined. A systems analyst determines system inputs and outputs by interviewing, observing users, and analyzing documents and other existing systems. The information captured for each data flow may be summarized using a form that contains the following information:

1. An optional identification number. Sometimes the ID is coded using a scheme to identify the system and the application in the system.

FIGURE 8.2

An online order form from World's Trend Catalog Division.

FIGURE 8.3

An example of a data flow description from World's Trend Catalog Division.

Data Flow Description	
ID _____	
Name <u>Customer Order</u>	
Description <u>Contains customer order information and is used to update the customer master and item files and to produce an order record.</u>	

Source <u>Customer</u>	Destination <u>Process 1</u>
Type of Data Flow ☐ File ☒ Screen ☐ Report ☐ Form ☐ Internal	
Data Structure Traveling with the Flow <u>Order Information</u>	Volume/Time <u>10/hour</u>
Comments <u>Order record information for one customer order. The order may be received by Web entry, email, FAX, or by the customer telephoning the order-processing department directly.</u>	

2. A unique descriptive name for the data flow. This name is the text that should appear on the diagram and be referenced in all descriptions using the data flow.
3. A general description of the data flow.
4. The source of the data flow. The source could be an external entity, a process, or a data flow coming from a data store.
5. The destination of the data flow (same items listed under the source).
6. An indication of whether the data flow is a record entering or leaving a file or a record containing a report, form, or screen. If the data flow contains data that are used between processes, it is designated as *internal*.
7. The name of the data structure describing the elements found in this data flow. For a simple data flow, it could be one or several elements.
8. The volume per unit of time. The data could be records per day or any other unit of time.
9. An area for further comments and notations about the data flow.

Once again, we can use our World's Trend Catalog Division example from Chapter 7 to illustrate a completed form. Figure 8.3 is an example of the data flow description representing the screen used to add a new CUSTOMER ORDER and to update the customer and item files. Notice that the external entity CUSTOMER is the source and that PROCESS 1 is the destination, providing linkage back to the DFD. The checked box for "Screen" indicates that the flow represents an input screen. It could be any screen, such as a web page, graphic user interface (GUI), mobile phone, or perhaps a mainframe screen. The detailed description of the data flow could appear on this form, or it could be represented as a data structure.

Data flows for all inputs and outputs should be described first because they usually represent the human interface, followed by the intermediate data flows and the data flows to and from data stores. The detail of each data flow is described using elements sometimes called fields, a data structure, or a group of elements.

A simple data flow may be described using a single element, such as a customer number used by an inquiry program to find the matching customer record.

Describing Data Structures

Data structures are usually described using algebraic notation. This method allows an analyst to produce a view of the elements that make up the data structure, along with information about those elements. For instance, the analyst will denote whether there are many of the same elements in the data structure (a repeating group) or whether two elements may exist mutually exclusive of each other. The algebraic notation uses the following symbols:

1. An equal sign (=) means “is composed of.”
2. A plus sign (+) means “and.”
3. Braces { } indicate repetitive elements, also called repeating groups or tables. There may be one repeating element or several in a group. The repeating group may have conditions, such as a fixed number of repetitions, or upper and lower limits for the number of repetitions.
4. Brackets [] are used for an either/or situation. Either one element or another may be present, but not both. The elements listed between the brackets are mutually exclusive.
5. Parentheses () are used for an optional element. Optional elements may be left blank on entry screens and may contain spaces or zeros for numeric fields in file structures.

Figure 8.4 is an example of the data structure for adding a customer order at World’s Trend Catalog Division. Each NEW CUSTOMER screen consists of the entries found on the right side of the equal signs. Some of the entries are elements, but others, such as CUSTOMER NAME, ADDRESS, and TELEPHONE, are groups of elements or structural records. For example, CUSTOMER NAME is made up of FIRST NAME, MIDDLE INITIAL, and LAST NAME. Each structural record must be further defined until the entire set is broken down into its component elements. Notice that following the definition for the CUSTOMER ORDER screen are definitions for each structural record. Even a field as simple as the TELEPHONE NUMBER is defined as a structure so that the area code may be processed individually.

Structural records and elements that are used in many different systems are given a non-system-specific name, such as street, city, and zip code, that does not reflect the functional area in which they are used. This method allows the analyst to define these records once and use them in many different applications. For example, a city may be a customer city, a supplier city, or an employee city. Notice the use of parentheses to indicate that (MIDDLE INITIAL), (APARTMENT), and (ZIP EXPANSION) are optional ORDER information (but not more than one). Indicate the OR condition by enclosing the options in square brackets and separating them with the symbol.

Logical and Physical Data Structures

When data structures are first defined, only the data elements the user would see, such as a name, address, and balance due, are included. This stage is the logical design, showing what data the business needs for its day-to-day operations. As we have learned from human–computer interaction (HCI), it is important that the logical design accurately reflect the mental model of how the user views the system. Using the logical design as a basis, the analyst then designs the physical data structures, which include additional elements necessary for implementing the system. The following are examples of physical design elements:

1. Key fields used to locate records in a database table. An example is an item number, which is not required for a business to function but is necessary for identifying and locating computer records.
2. Codes to identify the status of master records, such as whether an employee is active (currently employed) or inactive. Such codes can be maintained on files that produce tax information.
3. Transaction codes are used to identify types of records when a file contains different record types. An example is a credit file containing records for returned items as well as records of payments.
4. Repeating group entries containing a count of how many items are in the group.
5. Limits on the number of items in a repeated group.
6. A password used by a customer accessing a secure website.

Figure 8.5 is an example of the data structure for a CUSTOMER BILLING STATEMENT, one showing that the ORDER LINE is both a repeating item and a structural record. The ORDER

FIGURE 8.4

Data structure example for adding a customer order at World’s Trend Catalog Division.

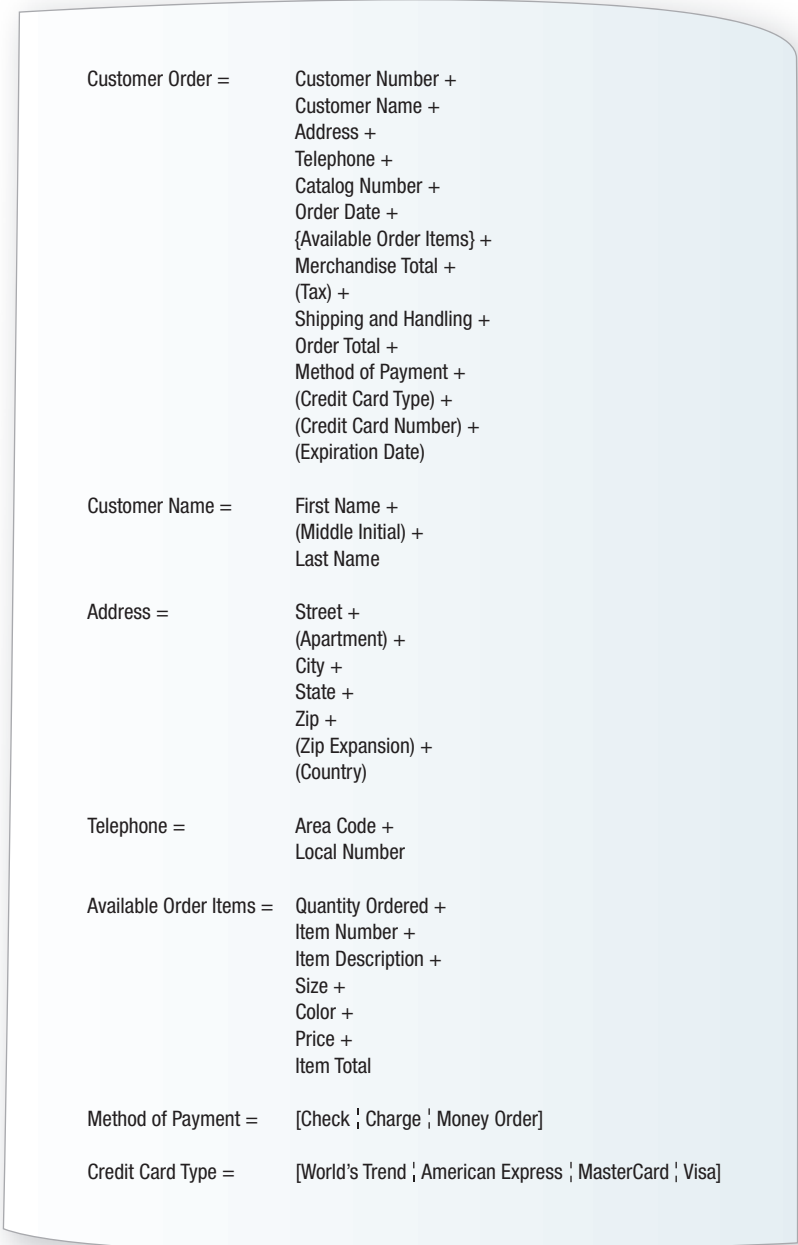
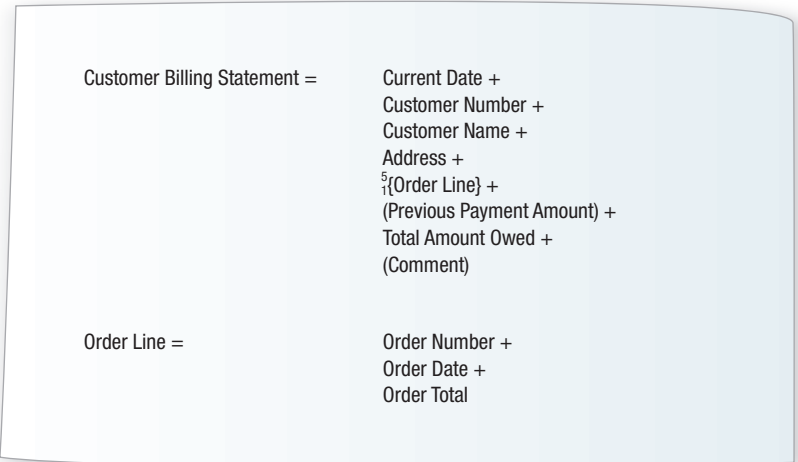


FIGURE 8.5

Physical elements added to a data structure.



LINE limits are from 1 to 5, indicating that the customer may order from one to five items on this screen. Additional items would appear on subsequent orders.

The repeating group notation may have several other formats. If the group repeats a fixed number of times, that number is placed next to the opening brace, as in 12 {Monthly Sales}, indicating the group repeats 12 times, once for each month in the year. If no number is indicated, the group repeats indefinitely. An example is a table containing an indefinite number of records, such as Customer Master Table = {Customer Records}.

The number of entries in repeating groups may depend on a condition, such as an entry on the Customer Master Record for each item ordered. This condition could be stored in the data dictionary as {Items Purchased} 5, where 5 is the number of items.

Data Elements

Each data element is defined once in the data dictionary and may be entered on an element description form as well, such as the one illustrated in Figure 8.6. Characteristics commonly included on the element description form are the following:

1. **Element ID.** This optional entry allows the analyst to build automated data dictionary entries.
2. **The name of the element.** The name should be descriptive, unique, and based on what the element is commonly called in most programs or by the major user of the element.
3. **Aliases, which are synonyms or other names for the element.** Aliases are names used by different users in different systems. For example, a CUSTOMER NUMBER may also be called a RECEIVABLE ACCOUNT NUMBER or a CLIENT NUMBER.

Element Description Form											
ID											
Name	Customer Number										
Alias	Client Number										
Alias	Receivable Account Number										
Description	Uniquely identifies a customer who has made any business transaction within the last five years.										
Element Characteristics											
Length	6	Dec. Pt.									
Input Format	9 (6)		☐ Alphabetic								
Output Format	9 (6)		☐ Alphanumeric								
Default Value			☐ Date								
☒ Continuous or	☐ Discrete		☒ Numeric								
			☐ Base or ☒ Derived								
Validation Criteria											
Continuous Upper Limit <999999 Lower Limit >0	<table style="width: 100%; border-collapse: collapse;"> <thead> <tr> <th style="width: 30%;">Discrete Value</th> <th>Meaning</th> </tr> </thead> <tbody> <tr><td style="border-bottom: 1px solid black;"></td><td style="border-bottom: 1px solid black;"></td></tr> <tr><td style="border-bottom: 1px solid black;"></td><td style="border-bottom: 1px solid black;"></td></tr> <tr><td style="border-bottom: 1px solid black;"></td><td style="border-bottom: 1px solid black;"></td></tr> </tbody> </table>			Discrete Value	Meaning						
Discrete Value	Meaning										
Comments The customer number must pass a modulus-11 check digit test. It is derived because it is computer generated and a check digit is added. 											

FIGURE 8.6

An element description form example from World's Trend Catalog Division.

4. **A short description of the element.**
5. **Whether the element is base or derived.** A base element is one that is initially keyed into the system, such as a customer name, address, or city. Base elements must be stored in files. Derived elements are created by processes as the result of a calculation or a series of decision-making statements.
6. **The length of an element.** Some elements have standard lengths. In the United States, for example, lengths for state name abbreviations, zip codes, and telephone numbers are all standard. For other elements, the lengths may vary, and the analyst and user community must jointly decide the final length, based on the following considerations:
 - a. Numeric amount lengths should be determined by figuring the largest number the amount will probably contain and then allowing reasonable room for expansion. Lengths designated for totals should be large enough to accommodate the sum of the numbers accumulated in them.
 - b. Name and address fields may be given lengths based on the following table. For example, a last name field of 11 characters will accommodate 98 percent of the last names in the United States.
 - c. For other fields, it is often useful to examine or sample historical data found in the organization to determine a suitable field length. If the element is too small, the data that need to be entered will be truncated. The analyst must decide how that will affect the system outputs. For example, if a customer's last name is truncated, mail would usually still be delivered; if an email address is truncated, however, it will be returned as not found. Common lengths of fields are shown in Figure 8.7.

<i>Field</i>	<i>Length</i>	<i>Percentage of Data That Will Fit (U.S.)</i>
Last Name	11	98
First Name	18	95
Company Name	20	95
Street	18	90
City	17	99

FIGURE 8.7

Some examples of data formats used in PC systems.

Data Type	Meaning
Bit	A value of 1 or 0, a true/false value
Char, varchar, text	Any alphanumeric character
Datetime, smalldatetime	Alphanumeric data, several formats
Decimal, numeric	Numeric data that are accurate to the least significant digit; can contain a whole and decimal portion
Float, real	Floating-point values that contain an approximate decimal value
Int, smallint, tinyint	Only integer (whole digit) data
Currency, money, smallmoney	Monetary numbers accurate to four decimal places
Binary, varbinary, image	Binary strings (sound, pictures, video)
Cursor, timestamp, uniqueidentifier	A value that is always unique within a database
Autonumber	A number that is always incremented by one when a record is added to a database table

Formatting Character	Meaning
X	May enter or display/print any character
9	Enter or display only numbers
Z	Display leading zeros as spaces
,	Insert commas into a numeric display
.	Insert a period into a numeric display
/	Insert slashes into a numeric display
-	Insert a hyphen into a numeric display
V	Indicate a decimal position (when the decimal point is not included)

FIGURE 8.8

Format character codes.

7. **The type of data: numeric, date, alphabetic, varchar, or character.** Character data is also referred to as alphanumeric or text data. Varchar (variable character) data may contain any number of characters, up to a limit set by the database software. When using varchar, specifying the length is optional. Several of these formats are shown in Figure 8.7. Character fields may contain a mixture of letters, numbers, and special characters. If the element is a date, its format—for example, MMDDYYYY—must be determined. If the element is numeric, its storage type should be determined.

Personal computer formats, such as currency, number, or scientific, depend on how the data will be used. Number formats are further defined as integer, long integer, single precision, double precision, and so on. There are many other types of formats used with PC systems. Unicode is a standardized coding system for defining graphic symbols, such as Chinese or Japanese characters. Unicode is described in greater detail in Chapter 15. There are three standard formats for mainframe computers: zoned decimal, packed decimal, and binary. The zoned decimal format is used for printing and displaying data. The packed decimal format is commonly used to save space on file layouts and for elements that require a high level of arithmetic to be performed on them. The binary format is suitable for the same purposes as the packed decimal format but is less commonly used.

8. **Input and output formats.** These should be included, and it is helpful to use special coding symbols to indicate how the data should be presented. These symbols and their uses are illustrated in Figure 8.8. Each symbol represents one character or digit. If the same character repeats several times, the character followed by a number in parentheses indicating how many times the character repeats is substituted for the group. For example, XXXXXXXX would be represented as X(8).
9. **Validation criteria.** Validation data ensures that accurate data are captured by the system. Elements are either discrete, meaning they have certain fixed values, or continuous, with a smooth range of values. Here are common editing criteria:
- A range of values is suitable for elements that contain continuous data. For example, in the United States a student grade point average may be from 0.00 through 4.00. If there is only an upper or lower bound to the data, a limit is used instead of a range.
 - A list of values is indicated if the data are discrete. Examples are codes representing the colors of items for sale in World's Trend's catalog.
 - A table of codes is suitable if the list of values is extensive (for example, state abbreviations, telephone country codes, or U.S. telephone area codes).
 - For key or index elements, a check digit is often included.
10. **Any default value the element may have.** The default value is displayed on entry screens and is used to reduce the amount of keying that the operator may have to do. Usually, several fields in each system have default values. When using GUI lists or drop-down lists, the default value is the one currently selected and highlighted. When using radio buttons, the option for the default value is selected, and when using check boxes, the default value (either “yes” or “no”) determines whether the check box will have an initial check in it.
11. **An additional comment or remarks area.** This might be used to indicate the format of the date, special validation that is required, the check digit method used (explained in Chapter 15), and so on.

Data element descriptions such as CUSTOMER NUMBER may be called CLIENT NUMBER elsewhere in the system (and perhaps old code written with this alias needs to be updated).

Another kind of data element is an alphabetic element. At World's Trend Catalog Division, codes are used to describe colors: for example, BL for blue, WH for white, and GR for green. When this element is implemented, a table will be needed for users to look up the meanings of these codes. (Coding is discussed further in Chapter 15.)

Data Stores

All base elements must be stored in the system. Derived elements, such as the employee year-to-date gross pay, may be stored in the system as well. Data stores are created for each different data entity being stored. That is, when data flow base elements are grouped together to form a structural record, a data store is created for each unique structural record.

Because a given data flow may only show part of the collective data that a structural record contains, you may have to examine many different data flow structures to arrive at a complete data store description.

Figure 8.9 is a typical form used to describe a data store. The information included on the form is as follows:

1. **The data store ID.** The ID is often a mandatory entry to prevent the analyst from storing redundant information. An example would be D1 for the CUSTOMER MASTER.
2. **The data store name.** It should be descriptive and unique.
3. **An alias for the table.** CLIENT MASTER can be an alias for the CUSTOMER MASTER.

FIGURE 8.9

An example of a data store description form for World's Trend Catalog Division.

Data Store Description Form	
ID	<u>D1</u>
Name	<u>Customer Master</u>
Alias	<u>Client Master</u>
Description	<u>Contains a record for each customer.</u>
Data Store Characteristics	
File Type	☒ Computer ☐ Manual
File Format	☒ Database ☐ Indexed ☐ Sequential ☐ Direct
Record Size (Characters):	<u>200</u>
Block Size:	<u>4000</u>
Number of Records: Maximum	<u>45,000</u>
Average:	<u>42,000</u>
Percent Growth per Year:	<u>6</u> %
Data Set Name	<u>Customer.MST</u>
Copy Member	<u>Custmast</u>
Data Structure	<u>Customer Record</u>
Primary Key	<u>Customer Number</u>
Secondary Keys	<u>Customer Name</u>
	<u>Zip</u>
	<u>Year-to-Date Amount Purchased</u>
Comments	<u>The Customer Master records are copied to a history file and purged if the customer has not purchased an item within the past five years. A customer may be retained even if he or she has not made a purchase by requesting a catalog.</u>

4. **A short description of the data store.**
5. **The file type, either computer or manual.**
6. **The format designates whether the file is a database table or a simple flat file.** (File formats are detailed in Chapter 13.)
7. **The maximum and average number of records on the file as well as the anticipated growth per year.** This information helps the analyst predict the amount of disk space required for the application and is necessary for hardware acquisition planning.
8. **The file or data set name.** In the initial design stages, this item may be left blank.
9. **The data structure name.** This name should be found in the data dictionary. A link to the elements for this data store should be provided.
10. **Primary and secondary keys.** These must be elements (or a combination of elements) found in the data structure. In the example, the CUSTOMER NUMBER is the primary key and should be unique. The CUSTOMER NAME, ZIP, and YEAR-TO-DATE AMOUNT PURCHASED are secondary keys used to control record sequencing on reports and to locate records directly. (Keys are discussed in Chapter 13.)
11. **Comments are used for information that does not fit into any of the above categories.** They may include update or backup timing, security, or other considerations.

Creating a Data Dictionary

Data dictionary entries may be created after the data flow diagram has been completed, or they may be constructed as the data flow diagram is being developed. The use of algebraic notation and structural records allows an analyst to develop the data dictionary and the DFDs using a top-down approach. For instance, the analyst may create a Diagram 0 data flow after the first few interviews and, at the same time, make the preliminary data dictionary entries. Typically, these entries consist of the data flow names found on the DFD and their corresponding data structures.

After conducting several additional interviews with users to learn the details of the system and the ways they interact with it, the analyst will expand the DFD and create the child diagrams. The data dictionary is then modified to include the new structural records and elements gleaned from further interviews, observation, and document analysis.

Each level of a DFD should use data appropriate for the level. Diagram 0 should include only forms, screens, reports, and records. As child diagrams are created, the data flow into and out of the processes becomes more and more detailed, including structural records and elements.

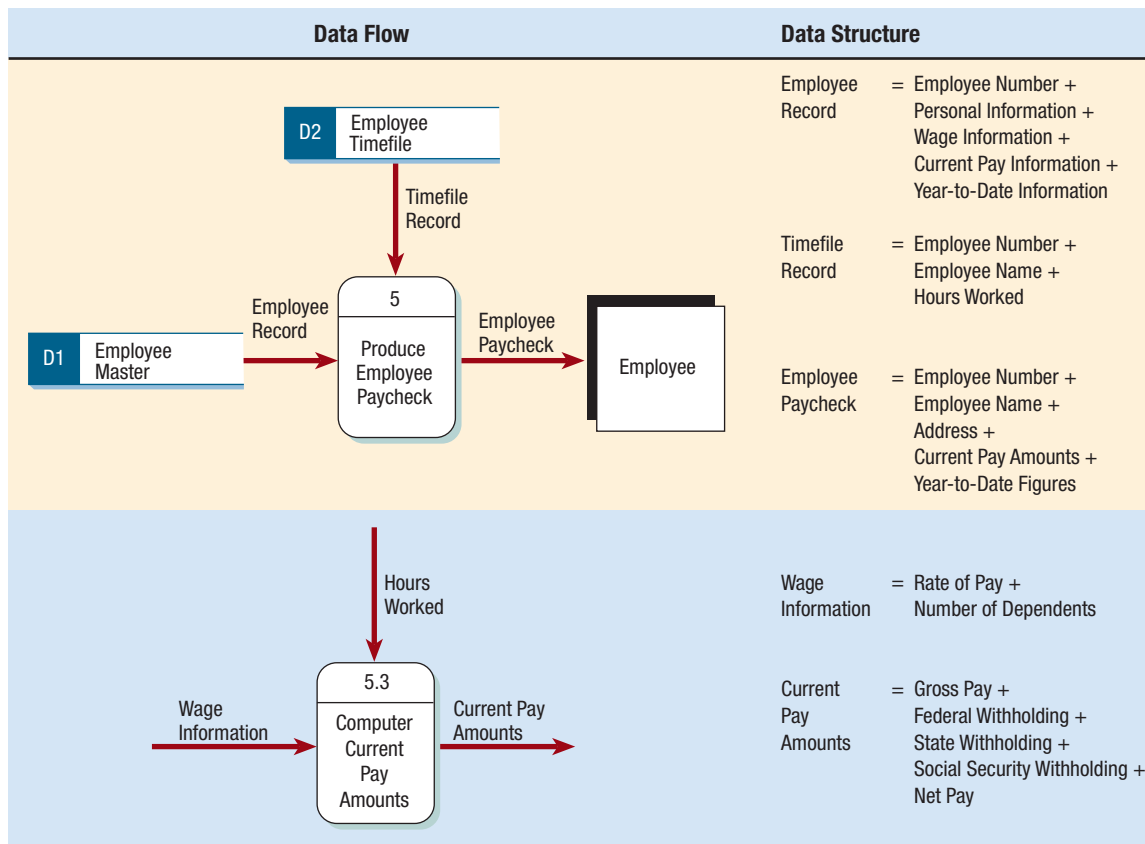
Figure 8.10 illustrates a portion of two DFD levels and corresponding data dictionary entries for producing an employee paycheck. Process 5, found on Diagram 0, is an overview of the production of an EMPLOYEE PAYCHECK. The corresponding data dictionary entry for EMPLOYEE RECORD shows the EMPLOYEE NUMBER and four structural records, data obtained early in the analysis. Similarly, TIMEFILE RECORD and the EMPLOYEE PAYCHECK also are defined as a series of structures.

It is important that the data flow names on the child DFD be contained as elements or structural records in the data flow on the parent process. Returning to the example, WAGE INFORMATION (input into process 5.3, COMPUTE CURRENT PAY AMOUNTS) is a structural record contained in the EMPLOYEE RECORD (input to process 5). Similarly, GROSS PAY (output from process 5.3.4, a lower-level process not shown in the figure) is contained in the structural record CURRENT PAY AMOUNTS (output from the parent process 5.3, COMPUTE CURRENT PAY AMOUNTS).

Analyzing Input and Output

An important step in creating a data dictionary is to identify and categorize system input and output data flow. Input and output analysis forms commonly include these fields:

1. A descriptive name for the input or output. If the data flow is on a logical diagram, the name should identify what the data are (for example, CUSTOMER INFORMATION). If the analyst is working on the physical design or if the user has explicitly stated the nature of the

**FIGURE 8.10**

Two data flow diagrams and corresponding data dictionary entries for producing an employee paycheck.

- input or output, the name should include that information regarding the format. Examples are CUSTOMER BILLING STATEMENT and CUSTOMER DETAILS INQUIRY.
- The user contact responsible for further details clarification, design feedback, and final approval.
- Whether the data is input or output.
- The format of the data flow. In the logical design stage, the format may be undetermined.
- Elements indicating the sequence of the data on a report or screen (perhaps in columns).
- A list of elements, including their names, lengths, and whether they are base or derived, and their editing criteria.

Once the form has been completed, each element should be analyzed to determine whether the element repeats, whether it is optional, or whether it is mutually exclusive of another element. Elements that fall into a group or that regularly combine with several other elements in many structures should be placed together in a structural record.

These considerations can be seen in the completed Input and Output Analysis Form for World's Trend Catalog Division shown in Figure 8.11. In this example of a CUSTOMER BILLING STATEMENT, the CUSTOMER FIRST NAME, CUSTOMER LAST NAME, and CUSTOMER MIDDLE INITIAL should be grouped together in a structural record.

Developing Data Stores

Another activity in creating a data dictionary is developing data stores. Up to now, we have determined what data need to flow from one process to another. This information is described in data structures. The information, however, may be stored in numerous places, and in each place the data store may be different. Whereas data flows represent data in motion, data stores represent data at rest.



CONSULTING OPPORTUNITY 8.1

Want to Make It Big in the Theatre? Improve Your Diction(ary)!

As you enter the door of Merman's, Annie Oaklea greets you warmly, saying, "I'm delighted with the work you have done on the data flow diagrams. I would like you to keep playing the role of systems analyst for Merman's and see if you can eventually get a new information system for our costume inventory sewn up. Unfortunately, some of the terms you're using don't come off very well in the language of Shakespeare. Bit of a translation problem, I suspect."

Clinging to Annie's initial praise, you are undaunted by her exit line. You determine that a data dictionary based on the rental and return data flow diagrams would make a big hit.

Begin by writing entries manually in as much detail as possible. Prepare two data flow entries, two data store entries, one data structure entry, and four data element entries using the formats in this chapter. Portraying interrelated data items with preciseness will result in rave reviews. (Refer to Consulting Opportunity 7.1.)

For example, when an order arrives at World's Trend (see Figure 8.12), it contains mostly temporary information—that is, the information needed to fill that particular order—but some information might be stored permanently. Examples of the latter include information about customers (so catalogs can be sent to them) and information about items (because these items will appear on many other customers' orders).

Input and Output Analysis Form			
Input/Output Name	<u>Customer Billing Statement</u>		
User Contact	<u>Susan Han</u>		
File Type	☒ Output	☐ Input	
File Format	☒ Report	☐ Screen	☐ Undetermined
Sequencing Element(s)	<u>Zip Code (Page Sequence)</u>		
	<u>Order Number</u>		
Element Name	Length	B/D	Edit Criteria
Current Date	6	B	(System Supplied)
Customer Number	6	D	(Includes Check Digit)
Customer First Name	20	B	Not Spaces
Customer Last Name	15	B	Not Spaces
Customer Middle Initial	1	B	A through Z or Space
Street	20	B	Not Spaces
Apartment	20	B	Not Spaces
City	20	B	Not Spaces
State	2	B	Valid State Abbr.
Zip	9	B	Numeric, Last 4 Opt.
Order Number	6	D	> 0
Order Date	8	B	MM/DD/YYYY
Order Total	9	D	Format: 9 (7) V99
Previous Payment Amount	5	D	Format: 9 (7) V99
Total Amount Owed	9	D	Format: 9 (7) V99
Comment	60	B	
Comments	<u>Print one page for each customer. If there are more items than will fit on a page, continue on a second page.</u>		

FIGURE 8.11

An example of an input–output analysis form for World's Trend Catalog Division.

FIGURE 8.12

Data stores derived from a pending order at World's Trend Catalog Division.

Customer Master =	Customer Number + Customer Name + Address + Telephone + Corporate Credit Card Number + Expiration Date
Item Master =	Item Number + Price + Quantity on Hand
Order Record =	Customer Number + Catalog Number + Order Date + {Available Order Items} + Merchandise Total + (Tax) + Shipping and Handling + Order Total + Method of Payment + (Credit Card Type) + (Credit Card Number) + (Expiration Date)
Available Order Items =	Item Number + Quantity Ordered + Quantity Shipped + Current Price
Method of Payment =	[Check Charge Money Order]
Credit Card Type =	[World's Trend American Express MasterCard Visa]

Data stores contain information of a permanent or semipermanent (temporary) nature. An ITEM NUMBER, DESCRIPTION, and ITEM COST are examples of information that is relatively permanent. So is the TAX RATE. When the ITEM COST is multiplied by the TAX RATE, however, the TAX CHARGED is calculated (or derived). Derived values do not have to be stored in a data store.

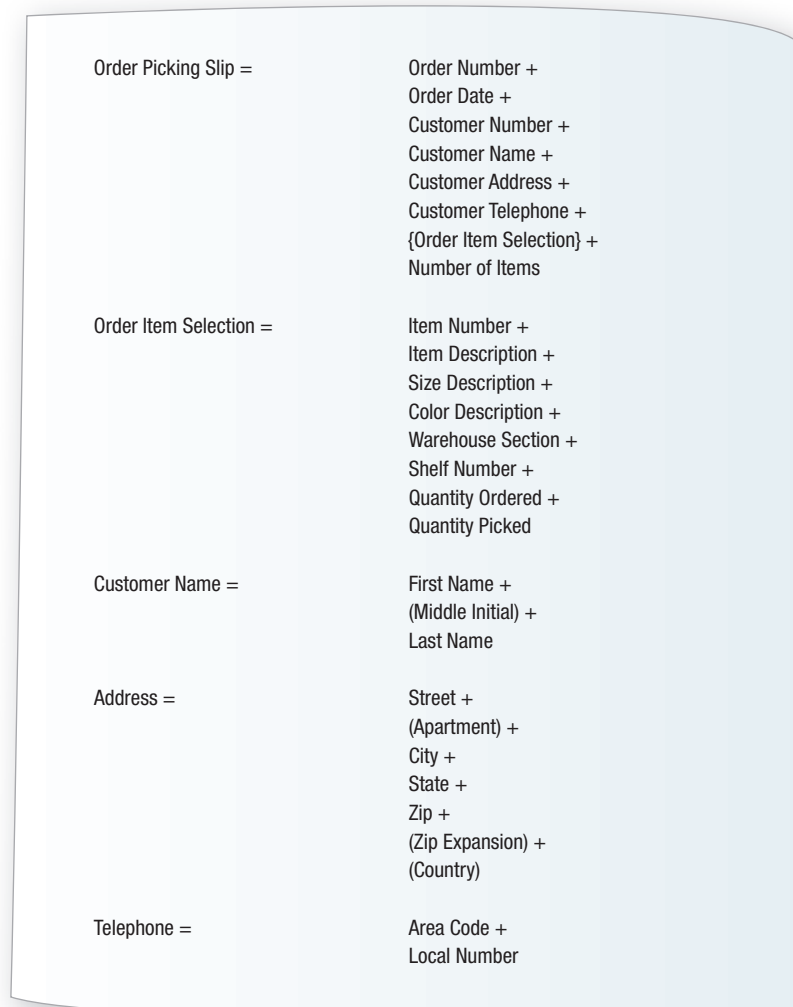
When data stores are created for only one report or screen, we refer to them as “user views” because they represent the way that the user wants to see the information.

Using a Data Dictionary

An ideal data dictionary is automated, interactive, online, and evolutionary. As a systems analyst learns about an organization's systems, he or she adds data items to the data dictionary. The data dictionary is not an end in itself, however, and must never become so. To avoid becoming side-tracked with building a complete data dictionary, a systems analyst should view it as an activity that parallels systems analysis and design.

To have maximum power, the data dictionary should be tied into a number of systems programs so that when an item is updated or deleted from the data dictionary, it is automatically updated or deleted from the database. The data dictionary becomes simply a historical curiosity if it is not kept current.

The data dictionary may be used to create screens, reports, and forms. For example, examine the data structure for the World's Trend ORDER PICKING SLIP in Figure 8.13.

**FIGURE 8.13**

Data structure for an order picking slip at World's Trend Catalog Division.

Because the necessary elements and their lengths have been defined, the process of creating physical documents consists of arranging the elements in a pleasing and functional way using design guidelines and common sense.

Repeating groups become columns, and structural records are grouped together on the screen, report, or form. The report layout for the World's Trend ORDER PICKING SLIP is shown in Figure 8.14. Notice that FIRST NAME and LAST NAME are grouped together in NAME and that QUANTITY (PICKED and ORDERED), SECTION, SHELF NUMBER, ITEM NUMBER, ITEM DESCRIPTION, SIZE, and COLOR form a series of columns because they are the repeating elements.

The data structure and elements for a data store are commonly used to generate corresponding computer language source code, which is then incorporated into computer programs. The data dictionary may be used in conjunction with a DFD to analyze the system design, detecting flaws and areas that need clarification. Some considerations are:

1. All base elements on an output data flow must be present on an input data flow to the process producing the output. Base elements are keyed and should never be created by a process.
2. A derived element must be created by a process and should be output from at least one process into which it is not input.
3. The elements that are present in a data flow coming into or going out of a data store must be contained in the data store.

FIGURE 8.14
Order picking slip created from
the data dictionary.



Order Picking Slip

Order Number: 999999

Customer Number: 999999

Order Date Z9/99/9999

Name: XXXXXXXXXXXXXXXXXXXXXXXXXXXX

Street: XXXXXXXXXXXXXXXXXXXXXXXXXXXX

Apartment: XXXXXXXX

City, State, Zip: XXXXXXXXXXXXXXXXXXXXXXXXXXXX, XX 99999-ZZZZ

Country: XXXXXXXXXXXXXXXXXXXXXXXXXXXX

Telephone: (999) 999-9999

--- Quantity ---		Section	Shelf Number	Item Number	Item Description	Size	Color
Picked	Ordered						
_____	ZZZZ9	XXXXX	99999	999999	XXXXXXXXXXXXXXXXXXXX	XXXXXXXXXX	XXXXXX
_____	ZZZZ9	XXXXX	99999	999999	XXXXXXXXXXXXXXXXXXXX	XXXXXXXXXX	XXXXXX
_____	ZZZZ9	XXXXX	99999	999999	XXXXXXXXXXXXXXXXXXXX	XXXXXXXXXX	XXXXXX
_____	ZZZZ9	XXXXX	99999	999999	XXXXXXXXXXXXXXXXXXXX	XXXXXXXXXX	XXXXXX
_____	ZZZZ9	XXXXX	99999	999999	XXXXXXXXXXXXXXXXXXXX	XXXXXXXXXX	XXXXXX
_____	ZZZZ9	XXXXX	99999	999999	XXXXXXXXXXXXXXXXXXXX	XXXXXXXXXX	XXXXXX
_____	ZZZZ9	XXXXX	99999	999999	XXXXXXXXXXXXXXXXXXXX	XXXXXXXXXX	XXXXXX
_____	ZZZZ9	XXXXX	99999	999999	XXXXXXXXXXXXXXXXXXXX	XXXXXXXXXX	XXXXXX

Number of Items: Z9

If begun early, a data dictionary can save many hours of time in the analysis and design phases. The data dictionary is the one common source in the organization for answering questions and settling disputes about any aspect of data definition. An up-to-date data dictionary can serve as an excellent reference for maintenance efforts on unfamiliar systems. Automated data dictionaries can serve as references for both people and programs.

Using Data Dictionaries to Create XML

Extensible markup language (XML) can be used to exchange data between businesses or between systems within a business. It is similar to HTML, the markup language used to create web pages, but XML is more powerful. HTML is concerned primarily with formatting a document; XML addresses the problem of sharing data when users have different computer systems and software or different database management systems (for example, one company using Oracle RDBMS and another using IBM’s DB2). If everyone used the same software or database management system, there would be little need for XML.

Once an XML document has been created, the data may be transformed into a number of different output formats and displayed in many different ways, including printed output, web pages, output for a handheld device, and portable document format (PDF) files. Thus, the document’s data content is separated from the output format. The XML content is defined once as data and then transformed as many times as necessary.

The advantage of using an XML document is that the analyst may select only the data that an internal department or external partner needs to have in order to function. This helps to ensure the confidentiality of data. For example, a shipping company may receive only the customer name, the address, the item number, and the quantity to ship, but *not* credit card information or other financial data. This efficient approach also cuts down on information overload.

Using XML, therefore, is a way to define, sort, filter, and translate data into a universal data language that can be used by anyone. XML may be created from databases, a form, or software programs; or it may be keyed directly into a document, a text editor, or an XML entry program.

The data dictionary is an ideal starting point for developing XML content. The key to using XML is creating a standard definition of the data. This is accomplished by using a set of tags or

Data Dictionary		XML
Customer =	Name + Address + Current Balance + {Order Information} + Payment	<pre> <?xml version="1.0"> <customers> <customer number="C15008"> <name type="I"> <lastname>Stadler</lastname> <firstname>Karen</firstname> <middle_initial>L</middle_initial> </name> <address> <street>123 Oak Street</street> <apartment>Suite 16</apartment> <city>Madison</city> <state>WI</state> <zip>43704</zip> <country>United States</country> </address> <current_balance>123.45</current_balance> <order customer_number="C15008"> <order_number>00123</order_number> <order_date format="yyyymmdd">2008-06-23</order_date> <ship_date format="yyyymmdd">2008-06-25</ship_date> <total>1345.89</total> </order> <order customer_number="C15008"> <order_number>00127</order_number> <order_date format="yyyymmdd">2008-09-18</order_date> <ship_date format="yyyymmdd">2008-09-26</ship_date> <total>240.00</total> </order> <payment> <check> <check_number>7234</check_number> </check> <payment_date format="yyyymmdd">2008-09-30</payment_date> <payment_amt>1585.89</payment_amt> </payment> </customer> </customers> </pre>
Name =	Last Name + First Name + (Middle Initial)	
Address =	Street + (Apartment) + City + State + Zip + Country	
Order Information =	Order Number + Order Date + Ship Date + Total	
Payment =	[Check Credit Card] + Payment Date + Payment Amt	
Check =	Check Number	
Credit Card =	Credit Card Number + Expiration Date	

FIGURE 8.15

Using a data dictionary entry to develop XML content. The XML document mirrors the data dictionary structure.

data names that are included before and after each data element or structure. The tags become the metadata, or data about the data. Data may be further subdivided into smaller elements and structures until all elements are defined. XML elements also may include attributes, additional data included within the tag that describe something about the XML element.

Figure 8.15 illustrates a data dictionary containing customer, order, and payment information. The overall collection of customers is included in what is called the root element: customers. An XML document may contain only one root element, so it is often the plural of the data contained in the XML document. Each customer may place many orders. The structure is defined in the two left columns, and the XML code appears on the right. CUSTOMER, as you can see, consists of a NAME, ADDRESS, CURRENT BALANCE, multiple ORDER INFORMATION entries, and a PAYMENT. Some of these structures are further subdivided.

The XML document tends to mirror the data dictionary structure. The first entry (other than an XML line identifying the document) is <customers>, which defines the entire collection of

customer information. The less than (<) and greater than (>) symbols are used to identify tag names (similar to HTML). The last line of the XML document is a closing tag, </customers>, signifying the end of the customer information.

Customer is defined first and contains an attribute, the customer number. There is often a discussion about whether data should be stored as an element or an attribute. In this case, they are stored as an attribute.

The name tag, <name>, is defined next because it is the first entry in the data dictionary. NAME is a structure consisting of LAST NAME, FIRST NAME, and an optional MIDDLE INITIAL. In the XML document, this structure starts with <name> and is followed by <lastname>, <firstname>, and <middle_initial>. Because spaces are not allowed in XML tag names, an underscore is typically used to separate words. The closing </name> tag signifies the end of the group of elements. Using a structure such as name saves time and coding if the transformation displays the full name. Each of the child elements will be on one line separated by a space. Name also contains an attribute, either I for individual or C for corporation.

Indentation is used to show which structures contain elements. Note that <address> is similar to <customer>, but when we get to <order_information> there is a big difference.

There are multiple entries for <order_information>, each containing an <order_number>, <order_date>, <shipping_date>, and <total>. Because the payment is made either by check or credit card, only one of these may be present. In our example, payment is by check. The dates have an attribute called format that indicates whether the date appears as month, day, year; year, month, day; or day, month, year. If a credit card is used to make a payment, a TYPE attribute contains either an M, V, A, D, or O, indicating the type of credit card (MasterCard, Visa, and so on).

XML Document Type Definitions

Often the element structure of XML content is defined using a document type definition (DTD). A DTD is used to determine whether the XML document content is valid—that is, whether it conforms to the order and type of data that must be present in the document. The DTD is easy to create and well supported by standard software. Once the DTD has been completed, it may be used to validate the XML document using standard XML tools. The DTD is easier to create if a data dictionary has been completed because the analyst has worked with users and made decisions on the structure of the data.

Figure 8.16 illustrates the document type definition for the Customer XML document. Keywords, such as !DOCTYPE, indicating the start of the DTD, must be in capital letters. !ELEMENT describes an element, and !ATTLIST describes an attribute, listing the element name followed by the attribute name. An element that has the keyword #PCDATA, for parsed character data, is a primitive element, not further defined. An element that has a series of other elements within parentheses means that they are child elements and must be in the order listed. The statement <!ELEMENT name (lastname, firstname, middle_initial?)> means that the name must have the last name followed by the first name followed by the middle initial.

The question mark after “middle_initial” means that the element is optional and may be left out of the document for a particular customer. A plus sign means that there are one or more repeatable elements. Customers must contain at least one customer tag but could contain many customer tags. An asterisk means that there is zero or more of the elements. Each customer may have zero to many orders. A vertical bar separates two or more child elements that are mutually exclusive. Payment contains either check or credit card as options.

The attribute list definition for a customer number contains a keyword ID (in uppercase letters). This means that the attribute number must appear only once in the XML document as an attribute for an element with an ID. It is somewhat similar to a primary key. The difference is that, if the document had several different elements, each with an ID attribute, the given ID (C15008 in this example) could appear only once. An ID must start with a letter or an underscore and cannot be solely a number. The reason behind putting the customer number as an ID is to ensure that it is not repeated in a longer document. The keyword #REQUIRED

FIGURE 8.16

A document type definition for the customer XML document.

```

<!DOCTYPE customers [
<!ELEMENT customers (customer) + >
<!ELEMENT customer (name, address, current_balance, order*) >
<!ATTLIST customer number ID #REQUIRED>
<!ELEMENT name (lastname, firstname, middle_initial?) >
<!ATTLIST name type (I|C) #REQUIRED>
<!ELEMENT lastname (#PCDATA) >
<!ELEMENT firstname (#PCDATA) >
<!ELEMENT middle_initial (#PCDATA) >
<!ELEMENT address (street, apartment?, city, state, zip, country) >
<!ELEMENT street (#PCDATA) >
<!ELEMENT apartment (#PCDATA) >
<!ELEMENT city (#PCDATA) >
<!ELEMENT state (#PCDATA) >
<!ELEMENT zip (#PCDATA) >
<!ELEMENT country (#PCDATA) >
<!ELEMENT current_balance (#PCDATA) >
<!ELEMENT order (order_number, order_date, ship_date, total) >
<!ATTLIST order customer_number IDREF #REQUIRED>
<!ELEMENT order_number (#PCDATA) >
<!ELEMENT order_date (#PCDATA) >
<!ATTLIST order_date format (mmddyyyy|yyyymmdd|ddmmyyyy) #REQUIRED>
<!ELEMENT payment (check|credit_card) >
<!ELEMENT check (check_number) >
<!ELEMENT credit_card (credit_card_number, expiration_date) >
<!ATTLIST credit_card type (M|V|A|D|O) #REQUIRED>
<!ELEMENT credit_card_number (#PCDATA) >
<!ELEMENT expiration_date (#PCDATA) >
<!ELEMENT payment_date (#PCDATA) >
<!ATTLIST payment_date format (mmddyyyy|yyyymmdd|ddmmyyyy) #REQUIRED>
<!ELEMENT payment_amt (#PCDATA) >
]>

```

means that the attribute must be present. A keyword of **#IMPLIED** means that the attribute is optional. A document also may have an **IDREF** attribute, which links one element with another that is an **ID**. The **ORDER** tag has a **customer_number** attribute defined as an **IDREF**, and the value **C15008** must be present in an **ID** somewhere in the document. An attribute list containing values in parentheses means that the attribute must contain one of the values. A DTD definition **<!ATTLIST credit_card type (M|V|A|D|O) #REQUIRED>** means that the credit card type must be either an **M**, **V**, **A**, **D**, or **O**.

XML Schemas

A schema is another, more precise way to define the content of an XML document. Schemas may include the exact number of times an element may occur as well as the type of data within elements, such as character or numeric values, including the length of the element, limits on the data, and the number of places to the left and right of a decimal number.

A data dictionary is an excellent starting point for developing an XML document and a document type of definition or schema. The advantage of using XML to define data is that, in the XML format, data are stored in a pure text format and not dependent on any proprietary software. The document may be easily validated and transformed into many different output formats.

Industry groups or organizations may be involved in defining an industry-specific XML structure so that all involved parties understand what the data mean. This is very important when an element name may have several meanings. An example is “state,” which may mean a postal state abbreviation or the state or status of an order or account. Examples of industry-specific XML document type definitions and schemas may be found at www.xml.org.



HYPERCASE EXPERIENCE 8

“You’re really doing very well. Mr. Evans says you’ve given him all sorts of new ideas for running the new department. That’s saying quite a lot, when you consider that he has a lot of his own ideas. By now I hope you’ve had a chance to speak with everyone you would like to: certainly Evans himself, Tom Ketcham, Daniel Hill, and Mr. Hyatt.”

“Mr. Hyatt is an elusive soul, isn’t he? I guess I didn’t meet him until well into my third year. I hope you get to find out about him much sooner. Oh, but when you do get to see him, he cuts quite a figure, doesn’t he? And those crazy airplanes. I’ve almost been conked on the head by one in the parking lot. But how can you get angry when it’s The Boss who’s flying it? He’s also got a secret—or should I say private—Asian meditation garden off his office suite. No, you’ll never see it on the building plans. You have to get to know him very well before he’ll show you that, but I would wager it’s the only one like it in Tennessee and maybe in the whole United States. He fell in love with the wonderful gardens he saw in Southeast Asia as a young man. It goes deeper

than that, however. Mr. Hyatt knows the value of contemplation and meditation. If he has an opinion, you can be sure it has been well thought through.”

HYPERCASE Questions

1. Briefly list the data elements that you have found on three different reports produced at MRE.
2. Based on your interviews with Mr. Evans and others, list the data elements that you believe you should add to the Management Unit’s project reporting systems to better capture important data on project status, project deadlines, and budget estimates.
3. Create a data dictionary entry for a new data store and a new data flow that you are suggesting based on your response to Question 2.
4. Suggest a list of new data elements that might be helpful to Jimmy Hyatt but are clearly not being made available to him currently.

GEMS Documentation: Data Flow - Mozilla Firefox

DATA FLOW DESCRIPTION	
Name: Assignment Inquiry Response	
Description: A screen containing all Assignment information	
Source: Process 6.3.4, Format Assignment Response (and parent processes)	Destination: External Entity, Team Leader
Type of data flow: Screen	Volume/Time: Several per hour, varies
Data Structure Traveling with the Flow:	
Resource Number + Resource Name + Resource Availability Code + Resource Fee Amount + Fee Basis + Task Number + Task Description + Task Calendar Duration + Task Completion Date + Assignment Duration + Assignment Start Date + Assignment Scheduled Duration + Assignment Scheduled Start Date + Assignment Percent Completed	
Comments: All codes should be turned into code meanings for display on the screen. The Assignment Inquiry data flow contains the input fields, which should be included on the Assignment Inquiry screen.	
Reference Back Print	

FIGURE 8.HC1

In HyperCase, you can look at the data dictionary kept at MRE.

Summary

Using a top-down approach, a systems analyst uses data flow diagrams to begin compiling a data dictionary, which is a reference work that contains data about data—or metadata—on all data processes, stores, flows, structures, and logical and physical elements in the system being studied. One way to begin is by including all data items from data flow diagrams. A data dictionary is a specialized application of the kinds of dictionaries used as references in everyday life. A data dictionary is a reference work of data about data (that is,

metadata). Systems analysts compile data dictionaries to guide them through analysis and design. A data dictionary is a document that collects and coordinates specific data terms, and it confirms what each term means to different people in the organization. A data dictionary may be used to (1) validate the data flow diagram for completeness and accuracy, (2) provide a starting point for developing screens and reports, (3) determine the contents of data stored in files, (4) develop the logic for data flow diagram processes, and (5) create XML.

Automated data dictionaries (part of the CASE tools) are valuable for their capacity to cross-reference data items, thereby allowing necessary program changes to all programs that share a common element. This feature supplants changing programs on a haphazard basis, and it prevents waiting until the program won't run because a change has not been implemented across all applications sharing the updated item. Clearly, automated data dictionaries are important for large systems that produce several thousand data elements requiring cataloging and cross-referencing.

A larger collection of project information is called a repository. CASE tools permit the analyst to create and share a repository with team members that may include information about data flows, stores, record structures, and elements; about procedural logic screen and report design; and about data relationships. A repository also may contain information about project requirements and final system deliverables and about project management.

Each entry in a data dictionary contains the item name, an English description, aliases, related data elements, the range, the length, encoding, and necessary editing information. A data dictionary is useful in all phases of analysis, design, and ultimately documentation because it is the authoritative source on how a data element is used and defined by users in the system. Many large systems feature computerized data dictionaries that cross-reference all programs in the database using a particular data element. The data dictionary also can be used to create XML, enabling businesses with different systems, software, or database management systems to exchange data.

Keywords and Phrases

base element	packed decimal
binary format	physical data structure
data dictionary	repeating group
data element	repeating item
data structure	repository
derived element	schema
document type definition (DTD)	structural record
extensible markup language (XML)	system deliverables
ID	varchar
IDREF	zoned decimal

Review Questions

1. Define the term *data dictionary*. Define *metadata*.
2. Explain the rationale behind the contents of an automated data dictionary.
3. What information is contained in the data repository?
4. Discuss the algebraic notation for data structures.
5. Discuss the contents of a form that can be used to summarize the information captured for a data flow.
6. What are the characteristics that are commonly included in a form used to describe data elements?
7. Why are structural records used?
8. What is the difference between logical and physical data structures?
9. Describe the difference between base and derived elements.
10. What are some of the criteria used for validating data captured in a system?
11. Describe the creation of a data dictionary from the analysis of input and output forms.
12. Why shouldn't compiling the data dictionary be viewed as an end in itself?
13. List the five main ways to use a data dictionary.
14. What does extensible markup language (XML) describe?
15. What is a document type definition?
16. How does a document type definition help to ensure that an XML document contains all necessary elements?
17. When should attributes be used in an XML document?
18. What does an ID attribute ensure?
19. What is an XML schema?

CRUISE AVAILABILITY			
MM/DD/YYYY			HH:MM
ENTER STARTING DATE Z9-ZZZ-9999			

CRUISE INFORMATION:			
CRUISE SHIP	XXXXXXXXXXXXXXXXXX		
LOCATION	XXXXXXXXXXXXXXXXXX		
STARTING DATE	Z9-ZZZ-9999	ENDING DATE	Z9-ZZZ-9999
NUMBER OF DAYS	ZZ9		
COST	ZZ, ZZ9.99		
DISCOUNTS ACCEPTED	XXXXXXXXXX	XXXXXXXXXX	XXXXXXXXXX
OPENINGS REMAINING	ZZZZ9		

FIGURE 8.EX2

A screen prototype to show cruise availability.

- Analyze the elements found on the PRODUCT-PART web page and create the data structure for the PRODUCT MASTER and the PART MASTER data stores.
- Which of the elements on the PRODUCT-PART web page are derived elements?
- The Pacific Holiday Company arranges cruise vacations of varying lengths at several locations. When customers call to check on the availability of a cruise, a CRUISE AVAILABILITY INQUIRY, illustrated in Figure 8.EX2, is used to supply them with information. Create the data dictionary structure for the CRUISE AVAILABILITY INQUIRY.
- List the master files that would be necessary to implement the CRUISE AVAILABILITY INQUIRY.
- The following ports of call are available for the Pacific Holiday Company:

Apia	Nuku Hiva	Auckland
Pago Pago	Papeete	Wellington
Bora Bora	Raiatea	Christ Church
Moorea	Napier	Dunedin

Create the PORT OF CALL element. Examine the data to determine the length and format of the element.

- Raúl Esparza, the ecommerce manager for Barba Bowls, a company that sells personalized sets of wooden salad bowls, would like to send information to another company that maintains the warehouse and provides shipping services. Order information is obtained from a secure website, including customer number, name and address, telephone number, email address, product number and quantity, as well as credit card information. There may be several different products shipped on one order. The shipping company handles items for other small businesses as well. Define an XML document that will include only the information that the shipping company needs to ship goods to the customer.
- Once the order in Problem 11 has been shipped, the shipping company sends information back to Barba Bowls, including the customer name and address, shipper tracking number, data shipped, quantity ordered, quantity shipped, and quantity backordered. Define an XML document that will include the information sent to Moonlight Mugs.
- Create a document type definition for Problem 11.
- Nine Lives Animal Rescue is a nonprofit organization that supports the fostering and adoption of animals, such as cats, dogs, and birds. People can register to adopt animals. Others register and add animals for adoption. Create the data dictionary structure representing a person registering to adopt an animal. Include name, address (street, city, state or province, zip or postal code), telephone number, email address, date of birth, current pets (type, breed, age of pet), and references. Each person may have multiple pets and must have at least three references. References must include name, address, telephone number, email address, and how they know the person registering to adopt an animal. Be sure to include notation for repeating elements and optional elements.
- Define the length, the type of data, and the validation criteria for each of the elements in Problem 14.
- List the data stores that would be required to implement the person registering in Problem 14.
- Create an XML document with sample data for one person registering to adopt an animal.

Group Projects

1. Meet with your group and use a CASE tool or a manual procedure to develop data dictionary entries for a process, data flow, data store, and data structure based on the data flow diagrams you completed for Maverick Transport in the Chapter 7 group exercises. As a group, agree on any assumptions necessary to make complete entries for each data element.
2. Your group should develop a list of methods to help you make complete data dictionary entries for this exercise as well as for future projects. For example, study existing reports, base them on new or existing data flow diagrams, and so on.

Selected Bibliography

- Agrawahl, A. "Building & Maintaining a Master Data Dictionary: Part One," 2016. <https://magento.com/blog/best-practices/building-maintaining-master-data-dictionary>. Last accessed August 28, 2017.
- Agrawahl, A. "Building & Maintaining a Master Data Dictionary: Part Two," 2017. <https://blog.rjmetrics.com/2017/01/19/building-maintaining-a-master-data-dictionary-part-two/>. Last accessed August 28, 2017.
- Baskerville, R., and J. Pries-Heje. "Short Cycle Time Systems Development." *Information Systems Journal* 14 (2004): 237–264.
- Conboy, K., and B. Fitzgerald. "Toward a Conceptual Framework of Agile Methods: A Study of Agility in Different Disciplines." *Proceedings of the 2004 ACM Workshop on Interdisciplinary Software Engineering Research (WISER)*, Newport Beach, California (November 5, 2004): 37–44.
- Davis, G. B., and M. H. Olson. *Management Information Systems, Conceptual Foundations, Structure, and Development*, 2d ed. New York, NY: McGraw-Hill, 1985.
- Gane, C., and T. Sarson. *Structured Systems Analysis and Design Tools and Techniques*. Englewood Cliffs, NJ: Prentice Hall, 1979.
- Hoffer, J. A., R. Venkataraman, and H. Topi. *Modern Database Management*, 12th ed. Upper Saddle River, NJ: Pearson, 2016.
- Kelly, B., N. Sheppard, J. Delasalle, M. Dewey, O. Stephens, G. Johnson, and O. S. Taylor. "Open Metrics for Open Repositories." In *OR2012: The 7th International Conference on Open Repositories*, Edinburgh, Scotland, July 9–13, 2012.
- Kruse, R. L. and D. R. Mehr. "Data Management for Prospective Research Studies Using SAS® Software." *BMC Medical Research Methodology* 8, 61 (2008). doi: 10.1186/147-2288-8-61.
- Lucas, H. *Information Systems Concepts for Management*, 5th ed. New York, NY: McGraw-Hill, 1994.
- Martin, J. *Strategic Data-Planning Methodologies*. Englewood Cliffs, NJ: Prentice Hall, 1982.
- Sagheb-Tehrani, M. "Expert Systems Development and Some Ideas of Design Process." *ACM SIGSOFT Software Engineering Notes* 30, 2 (March 2005): 1–5.
- Subramaniam, V., and A. Hunt. *Practices of an Agile Developer*. Raleigh, NC: Pragmatic Bookshelf, 2006.